

토스 NEXT ML CHALLENGE : 광고 클릭 예측(CTR) 모델 개발

알고리즘 | 채용 | 토스 | 정형 | CTR | 머신러닝 | 분류

상금 : 1,000 만원

🕒 2025.09.08 ~ 2025.10.13 09:59 + Google Calendar

👤 2,574명 📅 마감



참여중

[Private 1위] GPU Hill Climbing Ensemble for Merlin framework



NAJUNGHWAN



공동작성자

2025.10.13 11:10 497 조회

29

13

< 1

0

다들 대회 고생 많으셨습니다.

대규모 추천 시스템 데이터를 다루고 다른 분들과 치열하게 경쟁할 수 있다는 점에서 즐거운 대회였습니다.

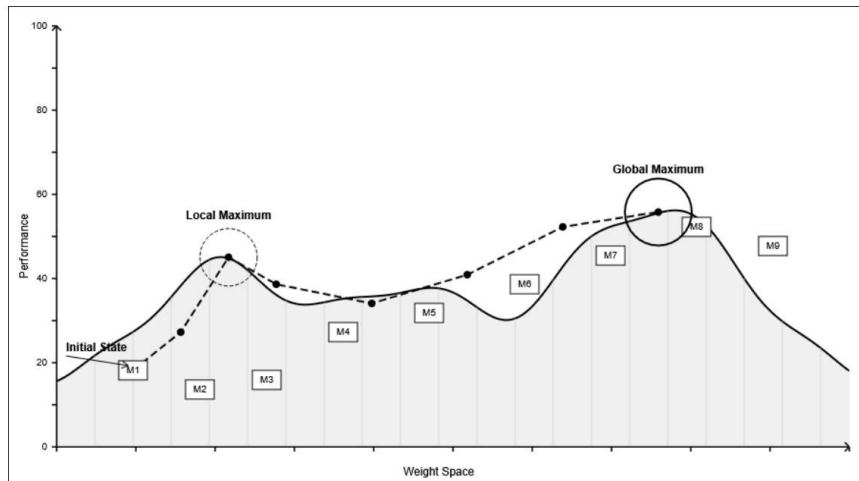
저희 팀 솔루션을 일부 공개합니다.

[전체 모델 아키텍처]

총 36개 모델의 hill-climbing 앙상블입니다.

가중치가 실제로 쓰인 건 25개 모델입니다.

GPU-Hill Climbing Optimization for Ensemble Model Weight Selection



Model Components:

M1 CatBoost (10 seeds)	M2 Logistic Regression (3 seeds)
M3 XGBoost Depthwise (10 seeds)	M4 XGBoost Lossguide (5 seeds)
M5 PyTorch MLP (3 seeds)	M6 xDeepFM (1 seed)
M7 PyTorch MLP over XGBoost (2 seeds)	M8 DCN v3 (1 seed)
M9 Wide & Deep (1 seed)	

[핵심 솔루션]

대회 안내 데이터 코드 공유 토크 리더보드 제출

대회 데이터셋을 시계열로 해석해 **sunday validation(temporal split)**을 했을 때 CV-LB gap이 가장 적었습니다.

(monday-saturday로 train, sunday로 val)

DAICON

커뮤니티

대회

학습

랭킹

더보기



구독 안내



sn



데이스를! 가을맞이 특별할인 [보러가기](#) >

이번 sunday validation을 통해 추천 시스템의 성능을 극대화하는 데 성공했습니다.

막상 LB는 "sunday가 train에 많이 포함될수록 높아진다"는 점을 발견했습니다.

예를 들어, friday를 validation으로 삼았을 때(평일과 주말의 중간 패턴) 단일 모델 LB가 가장 좋았습니다.

이때 CV-LB gap이 있지만, 상관관계는 안정적입니다. (sunday가 학습에 온전히 쓰였기 때문으로 해석됩니다)

[Friday validation CV-LB]

CV : 0.357602, LB : [0.3511151209](#)



CV : 0.357661, LB : 0.35125415
CV : 0.357699, LB : [0.3512969618](#)
CV : 0.357947, LB : [0.3517318686](#) (단일 모델 best)

이를 바탕으로
"test의 sunday 패턴을 맞춰야 하지만, 정작 sunday 패턴을 맞추려면 train에서 sunday를 학습해야 한다"
라는 결론을 얻었고, **domain adaptation task**라는 점을 확인하였습니다.

결과적으로 [day_of_week, clicked] 값이 각 fold에 모두 균등하게 들어가는 stratified-10fold CV로 확정했습니다.
이는 sunday를 train에 최대한 균등하게 넣으면서도, OOF 양상을 및 안정적인 LB 예측을 가능하게 합니다. (No negative sampling)

이때부터 CV-LB 갭은 있지만 차이가 안정적이므로, CV를 본격적으로 신뢰할 수 있었습니다.
다만 sunday 데이터가 fold로 인해 줄어든만큼, 단일 모델의 성능은 소폭 하락합니다. (약 -0.0001)

2. Merlin XGBoost with no FE

머신러닝에서 Feature Engineering을 할 때는 일반적으로 min_improvement를 톡니다.
통계적으로 "3-sigma rule"이라 하여 표준편차의 3배를 넘는 improvement를 채택하는 게 안정적입니다.
그러나 CTR 과제에서는 sparse-interaction과 signal/data ratio 문제가 있고,
설상가상으로 대규모 데이터일수록 개별 feature의 영향력은 통계적으로 약해집니다.

그렇다고 표준편차를 계산하지나 계산량이 기하급수적으로 늘어나는 문제가 있습니다.
예를 들어 5fold-cv의 CV가 0.353183 ± 0.000688 이런 식이라면,
안정적으로 채택할 수 있는 min_improvement는 약 + 0.002입니다. (약 99%의 신뢰도)

이런 improvement를 보이는 feature를 우리 대회에서는 찾을 수 없었고,
그렇다고 표준편차를 낮추기 위해 multiple-seed, 10fold 식으로 계산량을 기하급수로 늘릴 수도 없는 일입니다.
어찌보면 양자역학의 "불확정성의 원리"와도 비슷한 측면이 있다고 생각했습니다.

그렇다면 이런 상황에서는 검증하기 어렵고 효과가 미미한 Feature Engineering보다는,
확실하게 검증할 수 있는 모델 구조 및 파라미터 튜닝에 집중해야 한다는 결론을 얻었습니다.
또한 튜닝 이후에는 개별 모델의 예측을 보정에 줄 수 있는 multi-model ensemble이 핵심이라고 봤습니다.
실제 비슷한 대회 케이스를 경험했기에 빠른 판단이 가능했습니다.

Merlin framework로 1trial / 30초로 단축해 대규모 탐색이 가능했던 점이 이후 파이프라인 설계를 가능하게끔 했습니다.
[저희 솔루션에서 가장 자랑하고 싶은 부분입니다] (트리 모델 1fold 약 20초, pytorch mlp 1epoch 5초)

양상플에 쓰인 모든 모델의 개별 파라미터를 optuna로 최적화할 수 있었고,
가장 큰 특징은 scale_pos_weight를 별도의 optuna로 탐색해 정교한 최적값을 찾았다는 점입니다.
트리 모델의 경우 2.5~3.0
NN 모델의 경우 2.0~2.5이 최적이었습니다.

아래는 탐색한 단일 최고 모델 XGBoost_depthwise의 파라미터입니다.
Optuna는 LR=0.1로 고정해 빠르게 탐색하고, 본학습에서는 LR을 낮추면서 early stopping을 조정했습니다.
XGBoost가 가장 좋은 성능을 보였기에 해당 모델만 학습곡선을 모니터링하면서 LR을 미세조정하고,
나머지 모델들은 양상을 관점에서 모델 다양성이 중요하기에 LR 미세조정까진 하지 않았습니다.

```
scale_pos_weight=2.7314

Best Competition Score: 0.357581
Best hyperparameters:
max_depth: 15
max_leaves: 503
subsample: 0.8922288170923977
colsample_bytree: 0.6965797963245798
min_child_weight: 83
gamma: 1.0035755449048576
reg_alpha: 0.5576358212867057
reg_lambda: 0.8416579109274109
```

NN 모델의 경우 batch normalization 등의 아키텍처를 넣으면 최적 scale_pos_weight 값이 조금 더 낮아집니다. (약 1.7~1.8)
이는 BN이 input을 정규화해 gradient scale을 안정화하기 때문인데, 이 점이 오히려 혼란을 줄 수 있었습니다.

왜냐하면 scale_pos_weight=1로도 충분히 LB 0.348 정도가 나오기 때문에,
최적 scale_pos_weight 탐색을 미처 생각하지 못하게 만들기 때문입니다.

만약 scale_pos_weight를 건드리지 않았다면 negative sampling 등 다양한 기법을 사용해 이를 보정하는 솔루션을 사용했으리라 생각합니다.

아래 유튜브 영상을 참고했을 때,
토스에서 실제로 negative sampling 및 hard negative mining 기법을 사용하고 있기에,
negative sampling이 나쁘기보다는 이번 대회에서는 까다로운 기법이었다는 점을 말씀드리고 싶습니다.
(<https://www.youtube.com/watch?v=-jm8Gzmf3TM&t=884s>)
(이를테면, 실제 데이터 불균형이 약 51:1인데 scale_pos_weight=2가 최적이면 약 25:1로 negative sampling을 해야 비슷하기 때문입니다)

3.simple bi-GRU seq embedding

seq 전처리는 1024 길이로 뒤쪽을 truncate한 후, biGRU masking 방식의 사전학습을 수행했습니다.
이후 embedding projection 방식으로 32차원으로 줄여 32개 feature 형태로 모델에 주입했습니다.

이 방식을 택한 이유는 보고서에서 자세히 다루겠습니다.

한 가지 테크닉이 있다면,

원본 seq 열은 문자열이라 `cudf` 문자열 길이 제한에 막혀 `NVTabular` 통합이 어렵습니다.

하지만 이를 **정수열 리스트로 변환하면 NVTabular에 문제없이 통합됩니다.**

일반적으로 seq 열은 복잡한 패턴이 있기에 대부분의 논문에서 embedding 후 직접 NN 모델에 넣으라는 식의 권장사항이 있습니다.

이럴 경우 embedding 자체가 gradient에 들어가기에 표현력은 좋아지지만,

문제는 긴 seq 열이 학습시 매 배치마다 **backward pass**로 들어가 학습 속도가 매우 느려집니다.

이게 대부분의 참가자들이 고통받은 주요 원인으로 생각합니다.

(더군다나 GRU는 LSTM보다 간단한 구조임에도 sequential한 특성 탓에 병렬화가 어렵습니다)

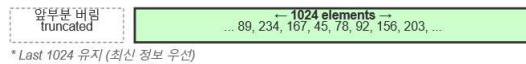
이 문제를 해결하기 위해 **embedding projection** 방식을 채택해 표현력을 어느정도 유지하면서도,

빠른 학습의 장점을 유지할 수 있었습니다.

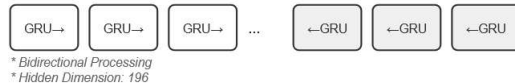
원본 가변 길이 시퀀스



뒤쪽 1024 길이 선택



BiGRU (Masking 적용)



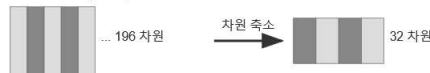
처리 단계 요약:

1. 뒤쪽 1024개만 선택 (앞부분 제거)
2. Bidirectional GRU 학습
3. Masking으로 패딩 처리
4. 196차원 → 32차원 축소

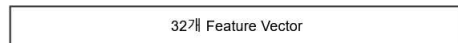
예시:

원본: 2000개 요소
→ 앞쪽 976개 버림
→ 뒤쪽 1024개 사용

Embedding Projection



최종 모델 입력



4.BCE / AUC early stopping with Temperature Scaling

일반적인 CTR 대회는 rank 또는 logloss만을 평가 metric으로 삼는 반면,

이번 대회는 **rank + logloss의 복합 메트릭**이었습니다.

이럴 경우 **logloss만으로 학습했을 때 rank가 망가지는 문제**가 있습니다.

그렇다고 AP를 직접적으로 최적화하기 위해 AUC를 학습에 사용할 순 없습니다. **(미분 불가능 문제)**

여러 참고 논문에서는 이를 BCE 학습 후 AUC early stopping 방식으로 해결하려고 권장하고 있습니다.

즉, 이를 통해 AP를 최대화한 후 **temperature scaling** 같은 후처리를 사용하면 됩니다.

간단히 설명드리면 학습은 **ranking** 위주로 하고, 이후 **후처리에서 ranking을 보존하면서 logloss를 최적화**하는 방식입니다.

단순 logloss로 학습할 경우 모델이 대부분의 예측을 0으로 찍는 현상이 나타납니다.

예를 들어, train의 CTR mean은 약 1.9%이므로

정상적인 학습이라면 val 예측도 약 1.9%가 돼야 합니다.

그러나 `scale_pos_weight`만큼 val 예측은 곱해진 값이 되고,

before calibration의 mean이 됩니다.

이 값은 이후 **temperature scaling** 때 competition score 최대화를 위해 **ranking을 보존**하면서도 최적 분포로 이동합니다.

(AP는 calibration 전후로 동일)

[`scale_pos_weight = 2.714`]

Before calibration: mean=0.053618, std=0.051298

After calibration: mean=0.427400, std=0.179013

5.GPU-Optimized Hill climbing Ensemble

Hill climbing ensemble는 여러 OOF 모델이 있을 때 해당 모델들의 최적 가중 평균을 찾는 기법입니다.

이 방식은 각 모델의 seed 개수에 상관없이 안정적으로 최적 조합을 찾고, 과적합에서 자유롭다는 장점이 있습니다.

각 모델의 영향력을 수치상으로 확인할 수 있다는 점에서도 큰 장점이 있습니다.

각 OOF 모델이 1000만행 x 2열에 달하므로(prediction, target),

`cudf`를 활용해 GPU 상에서 competition metric과 hill climbing을 빠르게 계산하는 방식으로 가속화되었습니다.

`scikit-learn`으로 AP를 계산하면 이 연산이 매우 느리므로, 반드시 `cupy` 등 GPU 상에서 계산하는 방식이 권장됩니다.

이때 ranking 계산을 위해 필연적으로 쓰이는 `argsort`를 제외하면 병목이 없어 아주 빠르게 계산됩니다.

Hill climbing은 본질적으로 완전 탐색이 아니라 local optima에 빠질 수 있으므로, random restart / shotgun 방식으로 여러 번 초기화 탐색을 진행합니다.

XGBoost로 기본 파이프라인을 구축한 후, FE가 없는 동일 파이프라인에서 모델 구조만 바꾸며 최대한 다양한 모델을 탐색할 수 있었습니다. 이때 한 모델의 multiple-seed보다는 다른 모델 구조일수록 성능 향상이 있었습니다.

팀 병합 전 저와 팀원이 리더보드 2, 3위를 각자의 솔루션으로 차지했고, 앙상블했을 때 큰 폭의 점수 향상이 있었습니다. 각 모델의 파라미터는 보고서에 상세히 기재하겠습니다.

최종 Hill Climbing Ensemble 모델 가중치

```
CV score: 0.359380
AP: 0.089676
WLL: 0.589612

Optimal weights:
cat_111: 0.008251
cat_123: 0.034841
cat_2024: 0.031689
cat_222: 0.000000
cat_333: 0.012475
cat_42: 0.049274
cat_444: 0.056228
cat_456: 0.015851
cat_555: 0.013261
cat_789: 0.000005
lr_123: 0.000000
lr_42: 0.000000
lr_456: 0.000000
xgb_depthwise_123: 0.014420
xgb_depthwise_1234: 0.033989
xgb_depthwise_2345: 0.016674
xgb_depthwise_3456: 0.012877
xgb_depthwise_42: 0.050391
xgb_depthwise_456: 0.001077
xgb_depthwise_4567: 0.115380
xgb_depthwise_5678: 0.020009
xgb_depthwise_789: 0.020488
xgb_depthwise_999: 0.090879
xgb_lossguide_123: 0.029578
xgb_lossguide_2024: 0.033974
xgb_lossguide_42: 0.000000
xgb_lossguide_456: 0.020046
xgb_lossguide_789: 0.000000
mlp_123: 0.000000
mlp_42: 0.000000
mlp_456: 0.000000
wide_deep_oof_5fold_seqGRU: 0.255027
mlp_over_xgb_123: 0.000000
mlp_over_xgb_456: 0.000000
xdeepFM_42: 0.056283
dcnv3_42: 0.007033
```

catboost 10seed
logistic regression 3seed
xgb_depthwise 10seed
xgb_lossguide 5seed
pytorch_mlp 3seed
xdeepFM 1seed
pytorch_mlp_over_XGBoost 2seed
DCN_v3 1seed
wide_deep 1seed (@신약보단마약이지)

> 빠른 xgb 파이프라인 구축으로 모델만 바뀌어가며 각 모델의 세밀한 optuna 최적화가 가능했고, 40개 이상의 모델 앙상블이 가능했습니다.

pytorch_mlp_over_XGBoost는 단일 최고 모델인 XGBoost의 OOF의 잔차를 pytorch_mlp 상에서 학습하는 일종의 스택킹 기법입니다. (잔차 학습)

구축해 놓은 NN 모델을 그대로 활용하는 방식으로,
학습 때 train OOF를 활용하고
추론 단계에서 submission을 생성할 때만 test 예측을 결합하는 방식입니다.
(test 데이터 활용 X)

원리는 아래와 같습니다.
pytorch_mlp 모델이 좀더 강력했다면 효과가 있었을 것 같은데 다소 아쉬움이 있습니다.
Kaggle Competition에서 배운 방법입니다.

[pytorch_mlp_over_XGBoost]

- 학습 단계 (test 정보 사용 X):
- XGBoost CV → OOF 생성
 - 잔차 = y_train - OOF
 - NN을 잔차로 학습 (이때 값이 음수가 되므로 BCE logit으로 학습)

추론 단계 :

4. NN으로 test 예측

5. XGBoost test + NN test = 잔차모델의 최종 test

궁금한 점 있으면 댓글 또는 메일(shaun0927@naver.com)로 남겨 주시면 확인 후 답변 드리겠습니다.

저희 팀 솔루션이 비즈니스 및 향후 CTR 예측 대회 참가자들에게 도움이 되길 바랍니다.

감사합니다!

29

공유하기

솔루션

댓글 13개

snorlaxdw

comment

0 / 1000

작성

신약보단마약이지 2025.10.13 11:12
고생하셨습니다. 24시간 쫓아시여..
이제 주무실 수 있는거죠?(물론 발표 자료 만들고)
답글 달기

하고 하고싶은거할거임 2025.10.13 11:26
많이 배워갑니다
답글 달기

NOWIMSONG 2025.10.13 11:27
좋은 정보 공유 감사 드립니다 수고하셨습니다.
답글 달기

과적합방지위원회 2025.10.13 11:27
정말 수고 많이 하셨습니다!
답글 달기

NAJUNGHWAN 2025.10.13 11:28
고생 많으셨습니다. 오프라인에서 뵈 수 있도록 보고서도 화이팅입니다!

가용성 2025.10.13 11:32
고생하셨습니다. 정말 우승하실 만한 자격이 있습니다
답글 달기

aiminions 2025.10.13 11:33
고생하셨습니다. 존경합니다.
답글 달기

dswook 2025.10.13 11:37
고생하셨습니다.
답글 달기

이종철 2025.10.13 11:37
고생하셨습니다! 많이 배워갑니다.
답글 달기

당근E 2025.10.13 13:21
많이 배워갑니다 감사합니다
답글 달기

CandleMaster 2025.10.13 13:44
훌륭한 성과와 잘 정리된 솔루션까지 너무 좋네요 🙌
축하드립니다!
답글 달기

최정명 2025.10.13 14:05
와 엄청 많은 시도를 해보신 것 같네요 ...
수고 하셨습니다.
day of week 에 따른 성능 변화를 체크해볼 생각을 못했네요
hill-climbing, Merlin framework 도 새롭게 알아갑니다~
답글 달기

다람이 2025.10.13 14:56
테스트가 dow 7로 쉬프트되어있는걸 대회 초반에 EDA하고 적용을 안하고 있었네요...훌륭한 분석과 적용입니다! 정말 많이 배워갑니다!
답글 달기



🔥 댓글 작성 창이 위치가 댓글 리스트 상단으로 이동했습니다!

☰ 목록으로

이전 글	시퀀스 데이터 처리방법 공유 (w/o embedding) 대회 - 토스 NEXT ML CHALLENGE : 광고 클릭 예측(CTR) 모델 개발	좋아요 0	조회 25	댓글 0	한 시간 전
현재 글	[Private 1위] GPU Hill Climbing Ensemble for Merlin framework 대회 - 토스 NEXT ML CHALLENGE : 광고 클릭 예측(CTR) 모델 개발	좋아요 29	조회 497	댓글 13	5시간 전
다음 글	Private 리더보드 공개 및 차후 일정 안내 대회 - 토스 NEXT ML CHALLENGE : 광고 클릭 예측(CTR) 모델 개발	좋아요 2	조회 182	댓글 3	6시간 전

DACON

AI 해커톤 플랫폼

이용약관

대회 주최 문의

데이콘 서비스 소개

개인정보 처리방침

교육 문의

데이콘 채용

💬 📷 📺 🗨️ 🌐 한국어

데이콘(주) | 대표 김국진 | 699-81-01021
통신판매업 신고번호: 제 2021-서울영등포-1704호
직접정보제공사업 신고번호: J1204020250004
서울특별시 영등포구 문영로 3 익스온벤처타워 901호
이메일 dacon@dacon.io | 전화번호: 070-4102-0545
Copyright © DACON Inc. All rights reserved